

Validation compétence Studi : Préparer le déploiement d'une application sécurisée

Application : Hublot – Tableau de bord DIRM Méditerranée

(dirm.mediterranee.developpement-durable.gouv.fr, Ministère chargé de la Mer et de la Pêche)

Déploiement en ligne : <https://dirmhublot.netlify.app>

Référentiel : Programme en vigueur le 21/02/2024 – Déploiement, DevOps, CI/CD, tests, sécurité.

1. Déploiement continu (CD) et hébergement

- **Application hébergée et accessible :** <https://dirmhublot.netlify.app>
 - **Pipeline de déploiement :** À chaque `git push` sur la branche `main`, Netlify déclenche automatiquement un build puis un déploiement (livraison continue).
 - **Configuration du déploiement :** Fichier `netlify.toml` à la racine du projet :
 - Commande de build : `npm run build`
 - Dossier publié : `build`
 - Règles de redirection (SPA, fonctions serverless).
 - **Documentation du processus :** Voir `HOSTING.md` (options Docker, build manuel, Netlify/Vercel) et `COMMENT_VOIR_LES_DONNEES_SUR_NETLIFY.md` pour la mise en œuvre sur Netlify.
-

2. Intégration continue (CI) et YAML

- **Workflow d'intégration continue :** workflow GitHub Actions « CI » (fichier `.github/workflows/ci.yml`, **Annexe 01**) :
 - déclenchement sur `main`, `staging` et pull requests ;
 - **ESLint**, `npm run test:run` (48 tests), `npm run audit:prod`, `npm run build`, **Playwright E2E** ;
 - CD cloud : Netlify (fichier `cd-netlify.yml` + gate required checks) ;
 - CD NAS : `cd-nas.yml` + `scripts/deploy-nas.sh` (semi-automatisé).

- **Synthèse** : `DEVOPS.md` , `CD_PIPELINES.md` , `MONITORING.md` , `ENVIRONNEMENT_STAGING.md` .
 - **Rédaction en YAML** : La pipeline CI est décrite en YAML (syntaxe et structure attendues dans le référentiel).
 - **Automatisation des tests en DevOps** : Batterie de **48 tests** unitaires (Vitest), exécutés automatiquement dans le workflow CI. Fichiers : `src/services/dataService.test.ts` (12), `src/utils/dataCalculations.test.ts` (20), `src/utils/security.test.ts` (15), `src/config/environment.test.ts` (1). Commande : `npm run test:run` .
-

3. Préparer le déploiement d'une application sécurisée

- **Authentification** : Page de connexion (identifiants configurés via variables d'environnement au build) ; accès aux données protégé après authentification.
 - **Données sensibles** : Fichiers sensibles (`.env` , `agents.json` , données Excel, etc.) sont exclus du dépôt via `.gitignore` ; pas de secrets committés.
 - **Documentation sécurité et déploiement** :
 - `DEPLOIEMENT_SECURISE.md` : étapes pour un déploiement sécurisé (HTTPS, Docker, firewall, etc.).
 - `CHECKLIST_SECURITE.md` : checklist avant mise en production (authentification, HTTPS, headers, Docker, sauvegardes).
 - `SECURITE.md` : mesures de sécurité implémentées dans l'application.
 - **Environnement de test** : séparation **DEV**, **TEST** (CI), **TEST local** (`check:env` , Docker :4173), **STAGING**, **PROD** — voir `ENVIRONNEMENT_TEST.md` , fichiers `.env.development` / `.env.test` , badge UI hors production.
 - **Scripts dans la démarche DevOps** :
 - Script de conversion principal : `scripts/convert_suivi_etpt_to_json.py` (préparation des données pour l'app).
 - Script d'évolution orchestré : `scripts/run-evolution-pipeline.sh` (conversion + contrôles + push optionnel).
 - Scripts npm : `npm run build` , `npm run dev` , `npm run data:evolution` (voir `README.md` et `scripts/README.md`).
-

4. Synthèse pour le jury

Attendu du référentiel	Élément dans le projet
Déploiement d'une application	Application en ligne : https://dirmhublot.netlify.app
Déploiement continu (CD)	Netlify : build + déploiement automatique à chaque push
Intégration continue (CI)	Workflow GitHub Actions – tests automatiques puis build
YAML	<code>netlify.toml</code> , <code>.github/workflows/ci.yml</code> , <code>cd-netlify.yml</code> , <code>cd-nas.yml</code>
Documentation du processus de déploiement	<code>DEVOPS.md</code> , <code>DOCUMENTATION_DEPLOIEMENT.md</code> , <code>HOSTING.md</code> , <code>README.md</code> (racine)
Application sécurisée	Authentification, <code>.gitignore</code> pour les secrets, docs sécurité
Scripts / automatisations	Scripts npm, Python (conversion), configuration Netlify
Environnement de test	<code>ENVIRONNEMENT_TEST.md</code> — DEV, TEST (CI), TEST local, STAGING, PROD ; <code>npm run check:env</code>
Enjeux des plans de test	<code>ENJEUX_PLAN_TEST.md</code> — risques, stratégie, pyramide, traçabilité
Méthodes Agile pour le développement logiciel	<code>AGILE_DEVOPS.md</code> — user stories, critères d'acceptation, DoD, lien CI/CD
Planifier efficacement les tests	<code>PLANIFIER_EFFICACEMENT_LES_TESTS.md</code> — méthode, priorités, pyramide, Definition of Done, commandes démo
Valider les résultats des tests	<code>VALIDER_RESULTATS_TESTS.md</code> — statuts Passé/Échec, preuves CI, rapport, DoD livraison

Attendu du référentiel	Élément dans le projet
Élaborer un scénario de test	<code>SCENARIOS_TEST.md</code> (+ Annexe 13) ; synthèse jury Rendus/SCENARIOS_TEST.md
Plan de test, scénarios, validation	<code>PLAN_TEST.md</code> + <code>RAPPORT_EXECUTION_TESTS.md</code> ; ST-F* / ST-SEC* ; 48 tests Vitest + CI
Scripts d'évolution dans la démarche DevOps	<code>SCRIPTS_EVOLUTION_DEVOPS.md</code> + <code>scripts/run-evolution-pipeline.sh</code> + <code>scripts/deploy-nas.sh</code>

Conclusion : Le projet Hublot permet de valider la compétence « Préparer le déploiement d'une application sécurisée » et les éléments associés du référentiel Studi (démarche DevOps, bases du déploiement automatique, CI/CD, YAML, documentation et sécurisation).